Laboratório de Engenharia de Software

INF1636 – PROGRAMAÇÃO ORIENTADA A OBJETOS

Departamento de Informática – PUC-Rio

Ivan Mathias Filho
ivan.mathias@puc-rio.br

Laboratório de Engenharia de Software

Programa – Capítulo 3

- Ponteiros e variáveis de objeto
- Construtores e Blocos de Inicialização
- Envio de Mensagens em Java
- Variáveis e Métodos Estáticos
- Blocos Estáticos

© LES / PUC-Rio

Programa – Capítulo 3

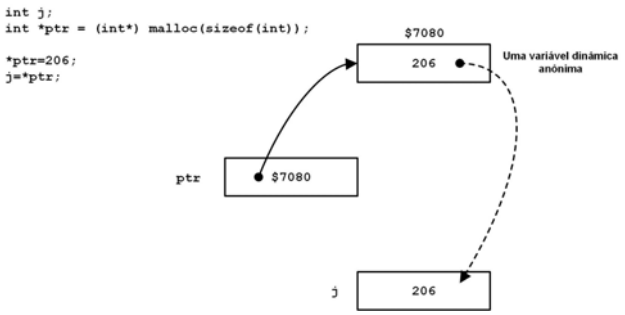
Laboratório de Engenharia de Software

- **Ponteiros e variáveis de objeto**
- Construtores e blocos de inicialização
- Envio de mensagens em Java
- Variáveis e métodos estáticos
- Blocos estáticos

Ponteiros

Laboratório de Engenharia de Software

- Um ponteiro é um tipo de dado cujo domínio é composto por endereços de memória e o valor **null**;
- O valor **null** não é um endereço de memória válido;
- Ele indica que, em dado momento, o ponteiro não pode ser usado para referenciar célula de memória alguma.



Ponteiros em Java (1)

Laboratório de Engenharia de Software

- A linguagem **Java** não possui ponteiros para tipos primitivos;
- Para manipular objetos em Java é preciso declarar **variáveis de objeto**;
- Uma **variável de objeto** é uma referência para um objeto;

Ponteiros em Java (2)

Laboratório de Engenharia de Software

- Enquanto um ponteiro em **C++** referencia uma área de memória, uma referência em **Java** referencia instâncias de classes;
- Como consequência, não se pode realizar operações aritméticas sobre referências em **Java**.

Criação de objetos

Laboratório de Engenharia de Software

- A simples declaração de uma variável de objeto não é suficiente para a criação de um objeto.

```
// v não referencia objeto algum
// o valor inicial de v é null

Vetor v=null;
```

- A criação de um objeto deve ser feita explicitamente, por meio do operador **new** (palavra reservada de **Java**).

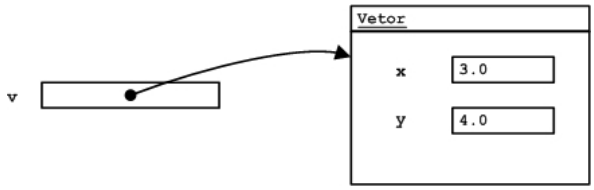
```
// criação de um objeto

v=new Vetor();
```


Alocação de objetos

Laboratório de Engenharia de Software

- O operador **new** aloca o objeto na área dinâmica de memória (**heap**) e retorna uma referência esse objeto;
- A referência é, então, atribuída à variável de objeto **v**.



Atribuição de variáveis de objeto

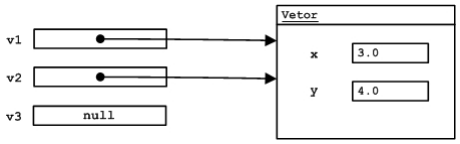
Laboratório de Engenharia de Software

- Como as variáveis de objeto são referências, as operações de atribuição entre elas não criam novos objetos.

```
Vetor v1,v2,v3;

v1=new Vetor();
v2=v1;
```

- Após a execução da atribuição **v2=v1**, ambas as variáveis irão referenciar o mesmo objeto.



Inicialização de variáveis de instância

Laboratório de Engenharia de Software

- É uma boa prática de programação inicializar as variáveis de instância de uma classe;
- Caso isso não seja feito, o compilador Java irá atribuir valores default às variáveis não inicializadas explicitamente. Isso será feito da seguinte maneira:
  - Variáveis numéricas (**byte**, **int**, **double** e etc.) serão inicializadas com o valor **zero**;
  - Variáveis do tipo **char** serão inicializadas com **\u0000**;
  - Variáveis do tipo **boolean** serão inicializadas com **false**;
  - Variáveis de objeto serão inicializadas com **null**.

Programa – Capítulo 3

Laboratório de Engenharia de Software

- Ponteiros e variáveis de objeto
- **Construtores e blocos de inicialização**
- Envio de mensagens em Java
- Variáveis e métodos estáticos
- Blocos estáticos

Construtores

Laboratório de Engenharia de Software

- Um construtor é um método especial que é executado todas as vezes que um objeto é criado;
- Ele é normalmente usado para fins de inicialização;
- Características de um construtor:
  - O nome de um construtor tem de ser igual ao nome da classe a qual ele pertence;
  - Um construtor possui zero ou mais parâmetros;
  - A chamada de um construtor é feita exclusivamente por meio do operador **new**;
  - Um construtor não retorna valor algum, embora seja incorreto usar o tipo **void** para indicar este fato.

Exemplo de construtor

Laboratório de Engenharia de Software

```
public class Contribuinte {
  private String nome;
  private String endereco;
  private String dtNasc;
  private long cpf;

  public Contribuinte() {  // construtor
    nome="João da Silva";
    dtNasc="01/01/1900";
  }

  public String getCpf() {
    // o código não é relevante
  }
}
```


Sobrecarga de método

Laboratório de Engenharia de Software

- Sobrecarregar (overloading) um método significa definir vários métodos com o mesmo nome, mas com diferentes parâmetros.

```
public class Teste {
  public void umMetodo() {
    // código irrelevante
  }
  public int umMetodo(int x) {
    return 2;
  }
  public void umMetodo(String y,char z) {
    // código irrelevante
  }
}
```

Sobrecarga de construtor

Laboratório de Engenharia de Software

- Como qualquer outro método, o construtor também pode ser sobrecarregado;
- Caso nenhum construtor seja definido, o compilador **Java** fornece um construtor default para a classe;
- O construtor default não possui parâmetros;
- Embora um construtor default possa ser inserido implicitamente pelo compilador **Java**, pode-se defini-lo explicitamente.

Construtor default - Exemplo

Laboratório de Engenharia de Software

```
public class Contribuinte {
  private String nome;
  private String endereco;
  private String dtNasc;
  private long cpf;

  public Contribuinte() {  // construtor default
    nome="João da Silva";
    dtNasc="01/01/1900";
  }

  public String getCpf() {
    // o código não é relevante
  }
}
```

Omissão do construtor default

Laboratório de Engenharia de Software

- Caso um construtor diferente do default seja definido, o compilador Java **NÃO** irá fornecer implicitamente um construtor default;
- Nesse caso, a invocação do construtor default, por meio do operador **new**, irá causar um erro de compilação.

```
public class Contribuinte {
  public Contribuinte(String n) {
    nome=n;
  }
}

...
{
 Contribuinte c=new Contribuinte(); // erro
}
```


Blocos de inicialização

Laboratório de Engenharia de Software

- Um bloco inicialização é um bloco de código não-nomeado definido no escopo de uma classe;
- Uma classe pode ter vários blocos de inicialização;
- Os blocos de inicialização são executados sempre que um objeto da classe na qual eles foram definidos é criado.

```
public class Contribuinte {
  // variáveis

  public Contribuinte() {
    nome="João da Silva";
    dtNasc="01/01/1900";
  }
  {
    cpf=0;
  }
}
```

Sequência de eventos na criação de objetos

Laboratório de Engenharia de Software

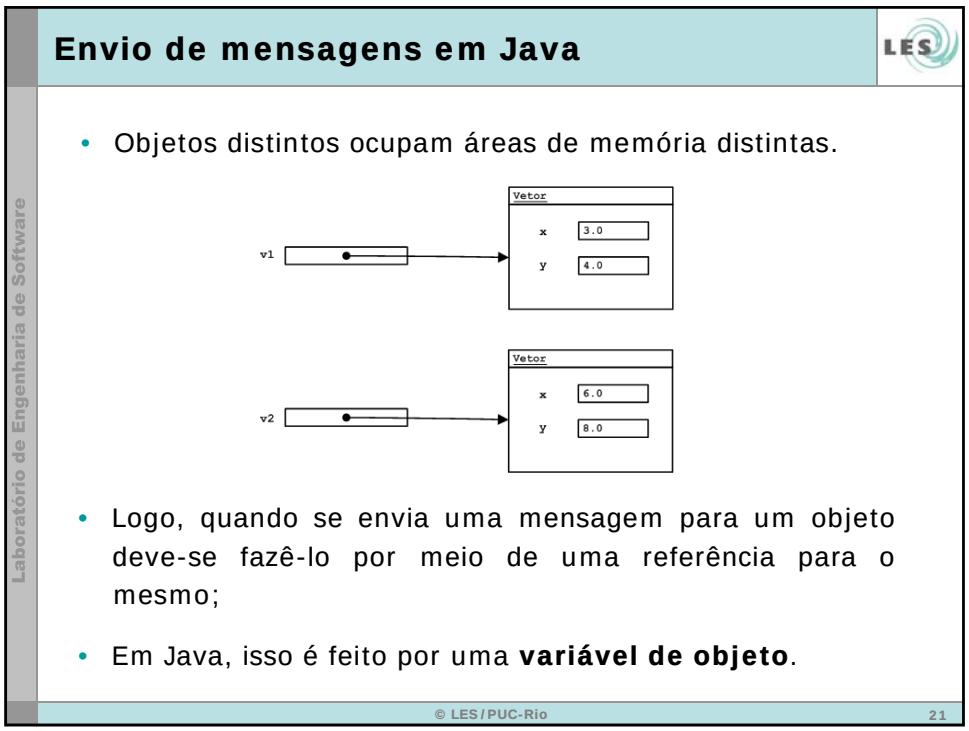
- Pode-se observar a seguinte sequência de eventos após a criação de um objeto:
  - Todas as variáveis de instância são inicializadas, na ordem em que foram declaradas;
  - Todos os blocos de inicialização são executados, na ordem em que foram definidos;
  - O construtor invocado pelo operador **new** é executado.

Programa – Capítulo 3

Laboratório de Engenharia de Software

- Ponteiros e variáveis de objeto
- Construtores e blocos de inicialização
- **Envio de mensagens em Java**
- Variáveis e métodos estáticos
- Blocos estáticos

Envio de mensagens em Java

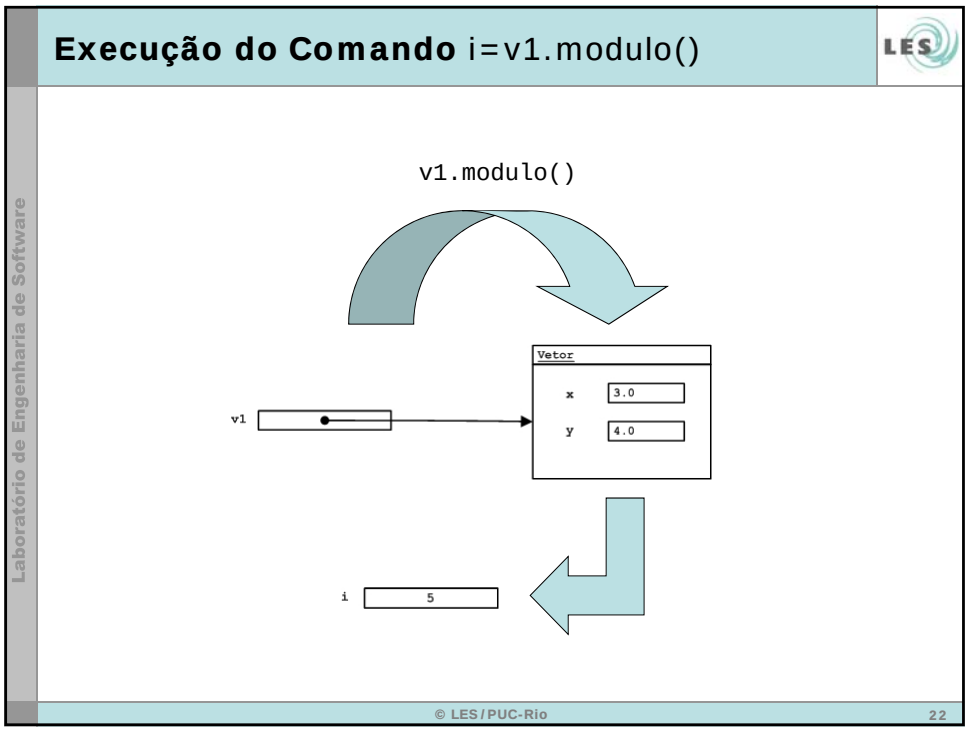


Laboratório de Engenharia de Software

- Objetos distintos ocupam áreas de memória distintas.

- Logo, quando se envia uma mensagem para um objeto deve-se fazê-lo por meio de uma referência para o mesmo;
- Em Java, isso é feito por uma **variável de objeto**.

Execução do Comando i=v1.modulo()



Laboratório de Engenharia de Software

Consequências

Laboratório de Engenharia de Software

- Para enviar mensagens para um objeto é necessário ter acesso a uma referência para o mesmo;
- Isso nos obriga a trafegar referências nas trocas de mensagens (parâmetros);
- Caso não haja pelo menos uma referência para um objeto ele se torna inacessível;
- Em Java, não é responsabilidade do programador liberar as áreas de memória inacessíveis;
- Uma thread que é executada em background, chamada **Garbage Collector**, reclama as áreas de memórias não-referenciadas e as devolve para a área de memória livre do heap.

Programa – Capítulo 3

Laboratório de Engenharia de Software

- Ponteiros e variáveis de objeto
- Construtores e blocos de inicialização
- Envio de mensagens em Java
- **Variáveis e métodos estáticos**
- Blocos estáticos

Variáveis estáticas

Laboratório de Engenharia de Software

- Em algumas situações é conveniente que um grupo de objetos de uma mesma classe tenha acesso a uma área de memória compartilhada;
- Como exemplo, pode-se citar a situação em que os objetos de uma classe têm de estar a par do número de objetos instanciados a partir dessa classe;
- Nesse caso, cada objeto instanciado deve somar uma unidade a uma variável que guarda o número de objetos instanciados até o momento;
- A linguagem Java oferece tal possibilidade por meio de **variáveis estáticas**.

Variáveis estáticas – Exemplo (1)

Laboratório de Engenharia de Software

- Uma variável estática pertence à classe, e não aos objetos da classe;
- Existe apenas uma instância de uma variável estática, independentemente do números de objetos instanciados.

```
public class Empregado {
  private int id;
  private double salario;
  private static int numEmp=0; // variável estática

  public Empregado() {
    numEmp++; //incrementa o número de empregados criados
  }
  public int getNumEmp() {
    return numEmp;
  }
}
```

Variáveis estáticas – Exemplo (2)

Laboratório de Engenharia de Software

```
public class Ex07 {
  public static void main(String[] args) {
    Empregado e1,e2,e3;

    e1=new Empregado();
    System.out.println(e1.getNumEmp());

    e2=new Empregado();
    System.out.println(e2.getNumEmp());

    e3=new Empregado();
    System.out.println(e3.getNumEmp());
  }
}
```

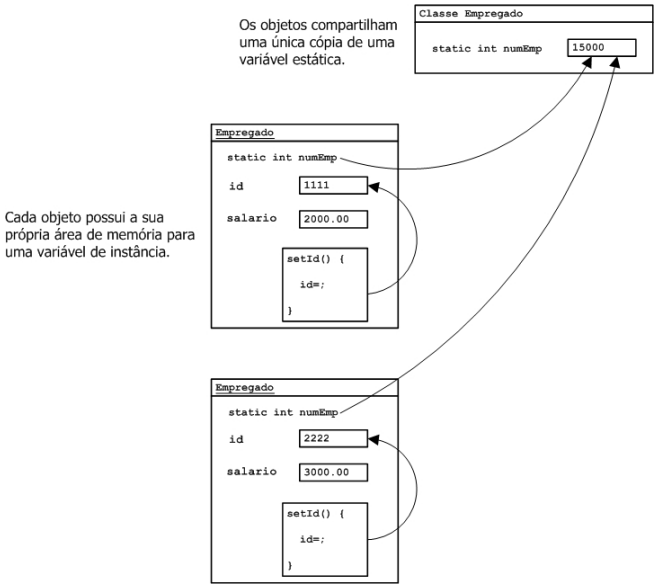
O valor **1** será exibido

O valor **2** será exibido

O valor **3** será exibido

Variáveis estáticas - Ilustração

Laboratório de Engenharia de Software



Métodos estáticos

Laboratório de Engenharia de Software

- Um método estático, ou método de classe, é um método que pertence à classe;
- Um método estático não pode manipular variáveis de instância, apenas as variáveis estáticas;
- A especificação da linguagem Java diz que um método de classe **deve** ser invocado sem referência para um objeto particular;
- Dessa forma, um método de classe deve ser invocado por meio do nome da classe.

```
a=Math.sqrt(Math.pow(x,2.0)+Math.pow(y,2.0));
```


Referência this

Laboratório de Engenharia de Software

- Todo objeto mantém uma referência para si mesmo;
- Esta referência é feita pela palavra reservada **this**.

```
public Contribuinte() {  // construtor
  this.nome="João da Silva";
  this.dtNasc="01/01/1900";
}
```

- No exemplo acima, o uso da referência **this** é desnecessário, pois sempre que se envia uma mensagem para um objeto deve-se fazê-lo por meio de uma referência para o mesmo;
- Logo, cada método de instância referencia, implicitamente, as variáveis de instância do objeto sobre o qual ele foi aplicado.

Referência this – Exemplo (1)

Laboratório de Engenharia de Software

- O uso da referência **this** é fundamental quando se deseja passar o próprio objeto como argumento em uma troca de mensagem.

```
public class Empregado {
  public void alocaGerente(Projeto p) {
    p.registraGerente(this);
  }
}

public class Projeto {
  Empregado gerente;

  public void registraGerente(Empregado e) {
    gerente=e;
  }
}
```


Referência this – Exemplo (2)

Laboratório de Engenharia de Software

```
public class Ex05 {
  public static void main(String[] args) {
    Projeto p=new Projeto();
    Empregado e=new Empregado();

    e.alocaGerente(p);
  }
}
```

Um novo projeto e um novo empregado são criados

A mensagem **alocaGerente** é enviada para o empregado (e) passando o projeto (p) como argumento

```
public class Empregado {
  public void alocaGerente(Projeto p) {
    p.registraGerente(this);
  }
}
```

O empregado envia a mensagem **registraGerente** para o projeto (p) passando ele próprio (this) como argumento

Referência this – Exemplo (3)

Laboratório de Engenharia de Software

```
public class Projeto {
  Empregado gerente;

  public void registraGerente(Empregado e) {
    gerente=e;
  }
}
```

O projeto registra o empregado recebido como parâmetro (e) como gerente do projeto

Referência this – Exemplo (4)

Laboratório de Engenharia de Software

- Pode-se usar a referência **this** para invocar um construtor a partir de outro construtor da mesma classe.

```
public class Vetor {
  // variaveis
  private double x=3.0;
  private double y=4.0;

  // métodos
  public Vetor(double a,double b) {
    x=a;
    y=b;
  }

  public Vetor() {
    this(0.0,0.0);
  }
}
```

O construtor default chama o construtor **Vetor(double, double)** passado o ponto (0.0,0.0) como argumento

Programa – Capítulo 3

Laboratório de Engenharia de Software

- Ponteiros e variáveis de objeto
- Construtores e blocos de inicialização
- Envio de mensagens em Java
- Variáveis e métodos estáticos
- **Blocos estáticos**

Blocos estáticos

Laboratório de Engenharia de Software

- Java permite definir blocos de código como estáticos (**static**);
- O código pertencente a um bloco estático é executado apenas quando a classe é carregada pela primeira vez;
- Tal como variáveis e métodos estáticos, os blocos estáticos estão no nível de classes;
- Portanto, os blocos estáticos são executados no momento do carregamento das classes às quais pertencem;
- Os blocos estáticos podem ser usados, por exemplo, para registrar os drivers de banco de dados e carregar bibliotecas nativas.

Blocos estáticos – informações adicionais

Laboratório de Engenharia de Software

- Blocos estáticos são executados pela JVM;
- Não é possível executar os blocos estáticos diretamente;
- Não é possível referenciar variáveis e métodos não-estáticos a partir de blocos estáticos;
- Os blocos estáticos são executados antes do construtor default;
- Não é possível definir níveis de acesso a blocos estáticos (não há blocos estáticos públicos, privados ou protegidos).

Fluxo de execução de blocos estáticos

Laboratório de Engenharia de Software

```
public class ExemploBlocoEstatico {
   {
      System.out.println("Bloco de inicialização");
   }
   static {
      System.out.println("Bloco estático 1");
   }
   public ExemploBlocoEstatico() {
      System.out.println("Construtor Default");
   }
   static {
      System.out.println("Bloco estático 2");
   }
   public static void main(String[] args) {
      ExemploBlocoEstatico blockDemo = new ExemploBlocoEstatico();
   }
   static {
      System.out.println("Bloco estático 3");
   }
}
```

Saída:

```
Bloco estático 1
Bloco estático 2
Bloco estático 3
Bloco de inicialização
Construtor Default
```